



ساختمان داده‌ها و الگوریتم‌ها

پاسخ تمرین چهارم - مرتب‌سازی و هش

محمدامین یوسفی، فرشته باقری

۱۵ نمره

۱. رشته با کمترین مقدار

فرض کنید مجموعه‌ی $S = \{s_1, s_2, \dots, s_n\}$ شامل n رشته از حروف کوچک انگلیسی داده شده است. طول طولانی‌ترین رشته در این مجموعه را با L نمایش می‌دهیم ($L = \max_i |s_i|$). هدف، یافتن یک جایگشت π روی اندیس‌های $\{1, \dots, n\}$ است تا رشته‌ی حاصل از چسباندن آن‌ها، یعنی $T = s_{\pi(1)} s_{\pi(2)} \dots s_{\pi(n)}$ ، از نظر لغت‌نامه‌ای (Lexicographically) کمینه باشد.

۱. طراحی الگوریتم: الگوریتمی کارا ارائه دهید که مسئله‌ی فوق را در زمان $O(n \cdot L \cdot \log n)$ حل کند. دقیقاً توضیح دهید که معیار شما برای تعیین ترتیب قرارگیری رشته‌ها چیست.

۲. اثبات صحت: ثابت کنید که الگوریتم پیشنهادی شما همواره پاسخ بهینه (کمینه) را تولید می‌کند. به بیان دقیق‌تر، نشان دهید که اگر خروجی الگوریتم شما رشته‌ی T_{opt} باشد، برای هر جایگشت دلخواه دیگر که منجر به رشته‌ی T' شود، رابطه‌ی $T_{opt} \leq_{lex} T'$ برقرار است.

پاسخ:

طراحی الگوریتم:

برای حل این مسئله، مرتب‌سازی رشته‌ها بر اساس ترتیب معمول لغت‌نامه‌ای راهگشا نیست. به عنوان مثال، اگر $A = "b"$ و $B = "ba"$ باشند، در ترتیب لغت‌نامه‌ای $A > B$ است، اما در حالت چسبیده، $A + B = "bba"$ بزرگتر از $B + A = "bab"$ خواهد بود. بنابراین باید معیار مقایسه را تغییر دهیم.

معیار صحیح مقایسه به این صورت تعریف می‌شود: برای هر دو رشته‌ی s_i و s_j ، رشته‌ی s_i باید قبل از s_j قرار گیرد اگر و تنها اگر $s_i + s_j \leq_{lex} s_j + s_i$ باشد که در آن $+$ نشان‌دهنده‌ی عمل چسباندن دو رشته است. الگوریتم پیشنهادی این است که با استفاده از یک روش مرتب‌سازی مانند Merge Sort یا Quick Sort، تمام رشته‌های مجموعه‌ی S را بر اساس این معیار خاص مرتب کرده و سپس رشته‌های مرتب‌شده را به ترتیب به یکدیگر بچسبانیم تا رشته‌ی نهایی T حاصل شود.

از نظر تحلیل پیچیدگی زمانی، چون تعداد رشته‌ها n است، مرتب‌سازی به $O(n \log n)$ مقایسه نیاز دارد. در هر گام مقایسه، دو رشته با طول حداکثر L به هم چسبانده و بررسی می‌شوند که هزینه‌ی آن $O(L)$ است. بنابراین پیچیدگی زمانی کل الگوریتم برابر با $O(n \cdot L \cdot \log n)$ خواهد بود.

اثبات صحت:

برای اثبات صحت، ابتدا باید نشان دهیم که رابطه‌ی تعریف‌شده یک رابطه‌ی ترتیب معتبر است، به ویژه اینکه خاصیت تراگذری (Transitivity) دارد (یعنی اگر $A + B < B + A$ و $B + C < C + B$ آنگاه $A + C < C + A$). برای این کار، هر رشته‌ی S را می‌توانیم به عنوان یک عدد در مبنای M (که M بزرگتر از تعداد حروف الفباست) در نظر بگیریم. اگر مقدار عددی رشته‌ی S را با $v(S)$ و طول آن را با $|S|$ نمایش دهیم، شرط $A + B < B + A$ معادل نابرابری ریاضی $v(A) \cdot M^{|B|} + v(B) < v(B) \cdot M^{|A|} + v(A)$ است. با بازآرایی این رابطه به فرم $\frac{v(A)}{M^{|A|-1}} < \frac{v(B)}{M^{|B|-1}}$ می‌رسیم. اگر تابع $f(S) = \frac{v(S)}{M^{|S|-1}}$ را تعریف کنیم، مقایسه‌ی رشته‌ها معادل مقایسه‌ی اعداد حقیقی $f(A)$ و $f(B)$ می‌شود. از آنجا که رابطه‌ی کوچکتر بودن در اعداد حقیقی تراگذری دارد، معیار مقایسه‌ی ما نیز تراگذری است.

در گام دوم برای اثبات بهینگی، از برهان خلف و استدلال Exchange Argument استفاده می‌کنیم. فرض کنید خروجی الگوریتم ما T_{alg} است و یک پاسخ بهینه‌ی دیگر T_{opt} وجود دارد که از نظر لغت‌نامه‌ای اکیداً کوچکتر است. چون T_{opt} ترتیب متفاوتی دارد، حتماً حداقل یک جفت رشته‌ی مجاور مانند (X, Y) در آن وجود دارد که Y بلافاصله پس از X آمده است، اما طبق معیار ما باید برعکس باشند (یعنی $Y + X < X + Y$). اگر در T_{opt} جای این دو رشته‌ی مجاور را عوض کنیم، رشته‌ی جدیدی حاصل می‌شود که از نظر لغت‌نامه‌ای کوچکتر (بتر) از T_{opt} است. این موضوع با فرض بهینه بودن T_{opt} در تناقض است. بنابراین، هیچ جفت مجاوری نمی‌تواند برخلاف معیار ما چیده شده باشد و ترتیب حاصل از الگوریتم پیشنهادی، پاسخ بهینه‌ی سراسری است.

۲. عنصر اکثریت

۱۵ نمره

فرض کنید آرایه‌ی A شامل n عدد صحیح $a_1, a_2, \dots, a_n$ و یک عدد صحیح x (به عنوان target) داده شده است. یک زیرآرایه‌ی پیوسته از A را با اندیس‌های شروع و پایان l و r مشخص می‌کنیم ($1 \leq l \leq r \leq n$). می‌خواهیم تعداد زیرآرایه‌هایی را بیابیم که در آن‌ها عدد x «عنصر اکثریت» باشد. عدد x در زیرآرایه‌ی $A[l \dots r]$ عنصر اکثریت است اگر و تنها اگر رابطه‌ی زیر برقرار باشد:

$$\text{count}(x, A[l \dots r]) > \frac{r - l + 1}{2}$$

که در آن count تعداد دفعات تکرار x در بازه‌ی مذکور است.

۱. الگوریتمی با پیچیدگی زمانی $O(n)$ طراحی کنید که تعداد این زیرآرایه‌ها را محاسبه کند.

۲. صحت الگوریتم خود را اثبات کنید.

پاسخ:

طراحی الگوریتم:

مسئله‌ی یافتن زیرآرایه‌هایی که در آن‌ها x عنصر اکثریت است را می‌توان به مسئله‌ی مجموع زیرآرایه‌ها تبدیل کرد. ابتدا آرایه‌ی جدید B به طول n را به صورت زیر می‌سازیم:

$$B[i] = \begin{cases} 1 & \text{if } A[i] = x \\ -1 & \text{if } A[i] \neq x \end{cases}$$

شرط اکثریت بودن x در بازه‌ی $[l, r]$ یعنی تعداد x ها بیشتر از نصف طول بازه باشد، که معادل است با اینکه مجموع عناصر B در این بازه مثبت باشد:

$$\sum_{k=l}^r B[k] > 0$$

حال اگر P را آرایه‌ی جمع جزئی (Prefix Sum) برای B تعریف کنیم ($P[0] = 0$ و $P[i] = P[i-1] + B[i]$)، شرط بالا معادل است با:

$$P[r] - P[l-1] > 0 \implies P[l-1] < P[r]$$

بنابراین مسئله تبدیل می‌شود به شمارش جفت‌های (i, j) که $0 \leq i < j \leq n$ و $P[i] < P[j]$. از آنجا که تغییرات P در هر گام دقیقاً $+1$ یا -1 است، می‌توانیم این شمارش را در زمان $O(n)$ انجام دهیم.

روی آرایه حرکت می‌کنیم و در هر لحظه تعداد جمع‌های قبلی که مقدارشان از جمع فعلی کمتر است را در متغیر `current_ans` نگه می‌داریم. همچنین از یک آرایه‌ی فرکانس `cnt` برای ذخیره‌ی تعداد دفعات مشاهده‌ی هر مقدار جمع پیشوندی استفاده می‌کنیم (با توجه به اینکه ممکن است مقادیر ممکن است منفی شوند، می‌توان از آفست n برای اندیس‌دهی استفاده کرد).

پیچیدگی زمانی این الگوریتم $O(n)$ است زیرا تنها یک بار روی آرایه حرکت می‌کنیم و تمامی عملیات‌های داخلی $O(1)$ هستند.

۳. بادکنک‌ها

۲۰ نمره

مجموعه‌ای از n بادکنک دراز داریم که هر کدام به صورت یک بازه‌ی عددی $[x_i, y_i]$ نمایش داده می‌شود. هر بادکنک در بازه‌ی بین x_i تا y_i روی محور اعداد قرار دارد. با پرتاب یک تیر در نقطه‌ی p (روی محور اعداد)، تمام بادکنک‌هایی که شرط زیر را ارضا کنند، می‌ترکند:

$$x_i \leq p \leq y_i$$

روشی برای یافتن کمترین تعداد تیری که کل بادکنک‌ها را بترکاند بیابید.

پاسخ:

ابتدا بازه‌ها را بر اساس نقطه پایان y_i به صورت صعودی مرتب می‌کنیم. سپس از اولین بازه شروع می‌کنیم و تیر اول را در نقطه $p = y_1$ (پایان اولین بازه) قرار می‌دهیم. هر بازه بعدی که شامل p باشد با همان تیر پوشش داده می‌شود؛ هرگاه به بازه‌ای برسیم که $x_i > p$ باشد یعنی توسط تیر فعلی پوشش داده نمی‌شود، یک تیر جدید در $p = y_i$ آن بازه می‌گذاریم و ادامه می‌دهیم. این انتخاب محلی — قرار دادن تیر در کوچک‌ترین مقدار ممکن انتها — بیشترین پوشش ممکن به سمت راست را فراهم می‌آورد و بنابراین حریصانه و معقول است.

اثبات بهینگی:

ادعا: الگوریتم بالا تعداد تیرها را به صورت بهینه (کمینه) محاسبه می‌کند.

اثبات: فرض کنید آرایه بازه‌ها پس از مرتب‌سازی بر حسب y برابر است با

$$I_1, I_2, \dots, I_n,$$

که $I_j = [x_j, y_j]$ و $y_1 \leq y_2 \leq \dots \leq y_n$. فرض کنید G مجموعه تیرهایی است که الگوریتم حریصانه در نقاط $g_1 < g_2 < \dots < g_k$ می‌گذارد و O مجموعه تیرهای یک جواب بهینه است که در نقاط $o_1 < o_2 < \dots < o_m$ قرار دارند. هدف نشان دادن این است که $k \leq m$ و در واقع $k = m$ وقتی O بهینه است.

در ادامه نشان می‌دهیم با تبدیل‌های محلی می‌توان O را طوری اصلاح کرد که حداقل k تیر داشته باشد و موقعیت تیرهایش از چپ به راست با g —ها هم‌راستا شوند؛ بنابراین نمی‌توان جواب بهینه‌ای با $k <$ تیر داشت.

قدم ۱ — مطابق اولین تیر: الگوریتم G اولین تیر را در $g_1 = y_r$ قرار می‌دهد، جایی که I_r اولین بازه در ترتیب مرتب‌شده است که تیر اول بر اساس آن گذاشته می‌شود (در واقع r کوچک‌ترین اندیسی است که هنوز پوشش نیافته). چون g_1 برابر پایان اولین بازه انتخاب‌شده است، هر بازه‌ای که الگوریتم با این تیر پوشش می‌دهد دارای $g_1 \leq x_j$ است.

در حالت O ، وجود دارد یک تیر اول o_1 . اگر $o_1 \geq g_1$ آنگاه می‌توانیم o_1 را به g_1 منتقل کنیم (یا به عبارت دیگر، تیر اول O را راست‌چین کنیم) بدون اینکه هیچ پوششی از دست برود، زیرا همه بازه‌هایی که با g_1 پوشش داده می‌شوند، نقاطی با شروع $g_1 \leq$ دارند و چون $o_1 \geq g_1$ هر بازه‌ای که با g_1 پوشش می‌یابد، با o_1 نیز پوشش داده می‌شده است یا بالعکس. اگر $o_1 < g_1$ ، چون g_1 انتخاب شد به این معنی است که حداقل یک بازه (مثلاً I_r) با $g_1 \leq x_r$ و $y_r = g_1$ وجود دارد که ممکن است توسط o_1 نیز پوشش داده شود؛ در این حالت نیز می‌توانیم o_1 را به g_1 منتقل کنیم بدون افزایش تعداد تیرها (حرکت تیر به راست پوشش موجود را از بین نمی‌برد زیرا g_1 از نظر پوشش نسبت به چپ آن بهتر یا برابر است). در نتیجه می‌توانیم فرض کنیم $o_1 = g_1$ پس از اصلاحات محلی.

قدم ۲ — استقرای موقعیت‌های بعدی: با حذف تمام بازه‌های پوشش داده‌شده توسط g_1 از مسئله، برای باقیمانده مسئله تکرار همین استدلال را داریم: الگوریتم G تیر بعدی را در g_2 می‌گذارد که پایان اولین بازه باقیمانده است؛ می‌توان به همان روش تیر دوم O را به g_2 منتقل کرد. با تکرار این تبدیل‌ها، می‌توانیم O را طوری تبدیل کنیم که تیرهایش در همان موقعیت‌های $g_1, g_2, \dots, g_k$ قرار گیرند و تعداد تیرها نه افزایش می‌یابد و نه کاهش می‌یابد.

بنابراین وجود یک جواب بهینه با تعداد کمتر از k تیر مستلزم وجود جوابی است که پس از تبدیل با تیرهای g برابر باشد، که ممکن نیست. لذا الگوریتم حریصانه مقدار کمینه تیرها را محاسبه می‌کند.

پیچیدگی زمانی: مرتب‌سازی بازه‌ها بر حسب y : $O(n \log n)$. پیمایش یک‌باره بازه‌ها برای شمارش تیرها: $O(n)$. بنابراین کل پیچیدگی زمانی $O(n \log n)$. فضای اضافی $O(1)$ به‌جز فضای لازم برای نگهداری ورودی یا $O(n)$ اگر نسخه مرتب‌شده جداگانه نیاز باشد.

۴. Linear Probing vs Double Hashing

۱۵ نمره

یک جدول هش با اندازه $m = ۱۳$ داریم و می‌خواهیم کلیدهای زیر را از چپ به راست در جدول قرار دهیم:

۱۸، ۳۱، ۴۴، ۵۷، ۷۰، ۸۳، ۹۶

(الف) برای هر یک از کلیدهای بالا، مراحل درج آن‌ها را با توجه به توابع هش زیر با هر دو روش Linear Probing و Double Hashing به صورت مرحله به مرحله نشان دهید و جدول نهایی را نیز رسم کنید. تابع هش در Linear Probing همان تابع اولیه Double Hashing است.

$$h_1(k) = k \bmod ۱۳, \quad h_2(k) = ۱ + (k \bmod ۱۲)$$

(ب) برای هر روش، تعداد کل تصادم‌ها را حساب کنید.

(ج) فرض کنید اجازه داریم ترتیب درج مجموعه کلیدها را به دلخواه خود عوض کنیم. آیا برای این مجموعه خاص می‌توانیم با عوض کردن ترتیب درج، تعداد تصادم‌ها را کم کنیم؟

پاسخ:

توجه: برای همه کلیدها $h_1(k) = k \bmod ۱۳ = ۵$ است.

روش اول: Linear Probing

فرمول: $h(k, i) = (۵ + i) \bmod ۱۳$

کلید	h_1	مراحل کاوش	محل نهایی
۱۸	۵	۵ (خالی)	۵
۳۱	۵	۵ (پر) ← ۶	۶
۴۴	۵	۵ (پر) ← ۶ (پر) ← ۷	۷
۵۷	۵	۵ → ۷ ← ۶ ← ۵	۸
۷۰	۵	۹ ← ۸ ← ۷ ← ۶ ← ۵	۹
۸۳	۵	۱۰ ← ۹ ← ۸ ← ۷ ← ۶ ← ۵	۱۰
۹۶	۵	۱۱ ← ۱۰ ← ۹ ← ۸ ← ۷ ← ۶ ← ۵	۱۱

جدول نهایی Linear Probing:

۱۲	۱۱	۱۰	۹	۸	۷	۶	۵	۴	۳	۲	۱	۰
	۹۶	۸۳	۷۰	۵۷	۴۴	۳۱	۱۸					

روش دوم: Double Hashing

فرمول: $h(k, i) = (۵ + i \cdot h_2(k)) \bmod ۱۳$

مقادیر: $h_2(k) = ۱ + (k \bmod ۱۲)$

$$h_2(۱۸) = ۱ + ۶ = ۷ \bullet$$

$$h_2(۳۱) = ۱ + ۷ = ۸ \bullet$$

$$h_2(۴۴) = ۱ + ۸ = ۹ \bullet$$

$$h_2(۵۷) = ۱ + ۹ = ۱۰ \bullet$$

$$h_2(۷۰) = ۱ + ۱۰ = ۱۱ \bullet$$

$$h_2(83) = 1 + 11 = 12 \bullet$$

$$h_2(96) = 1 + 0 = 1 \bullet$$

مراحل درج (همه ابتدا سراغ خانه ۵ می‌روند):

کلید	h_1	h_2	مراحل کاوش	محل نهایی
۱۸	۵	۷	۵ (خالی)	۵
۳۱	۵	۸	۵ (پر) $0 \cdot 13 = 0$ $(5 + 8) \cdot 13 = 0$ $\leftarrow^{+8}$ (خالی)	۰
۴۴	۵	۹	۵ (پر) $1 \cdot 13 = 1$ $(5 + 9) \cdot 13 = 1$ $\leftarrow^{+9}$ (خالی)	۱
۵۷	۵	۱۰	۵ (پر) $2 \cdot 13 = 2$ $(5 + 10) \cdot 13 = 2$ $\leftarrow^{+10}$ (خالی)	۲
۷۰	۵	۱۱	۵ (پر) $3 \cdot 13 = 3$ $(5 + 11) \cdot 13 = 3$ $\leftarrow^{+11}$ (خالی)	۳
۸۳	۵	۱۲	۵ (پر) $4 \cdot 13 = 4$ $(5 + 12) \cdot 13 = 4$ $\leftarrow^{+12}$ (خالی)	۴
۹۶	۵	۱	۵ (پر) $6 \cdot 13 = 6$ $(5 + 1) \cdot 13 = 6$ $\leftarrow^{+1}$ (خالی)	۶

جدول نهایی Double Hashing:

۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲
۳۱	۴۴	۵۷	۷۰	۸۳	۱۸	۹۶						

(ب) محاسبه تعداد کل تصادم‌ها

- در Linear Probing: کلید ۱۸: ۰ تصادم. کلید ۳۱: ۱ تصادم (با ۵). کلید ۴۴: ۲ تصادم (با ۵ و ۶). ... کلید ۹۶: ۶ تصادم.

$$\text{Total} = 0 + 1 + 2 + 3 + 4 + 5 + 6 = 21$$

- در Double Hashing: کلید ۱۸: ۰ تصادم. سایر ۶ کلید: هر کدام دقیقاً ۱ بار با خانه ۵ تصادم کردند و سپس در جای خالی نشستند.

$$\text{Total} = 0 + (6 \times 1) = 6$$

(ج) تغییر ترتیب درج

خیر، در این مثال خاص تعداد تصادم‌ها کاهش نمی‌یابد.

دلیل: چون $h_1(k)$ برای تمام کلیدهای این مجموعه برابر با ۵ است، همیشه اولین کلیدی که وارد شود خانه ۵ را می‌گیرد و ۶ کلید دیگر مجبورند حداقل یک بار با خانه ۵ برخورد کنند.

- در Double Hashing ما به کمترین مقدار ممکن (۶ تصادم) رسیده‌ایم (هر کلید فقط یک بار به ۵ می‌خورد و می‌رود). ترتیب درج فقط جای نهایی آن‌ها را عوض می‌کند اما تصادم اولیه‌شان با ۵ اجتناب‌ناپذیر است.

- در Linear Probing نیز چون همه روی ۵ جمع می‌شوند، هر ترتیبی که داشته باشند، باز هم یک "تپه" به طول ۷ از خانه ۵ تا ۱۱ تشکیل می‌دهند و مجموع تصادم‌ها همان ۲۱ باقی می‌ماند.

۵. ذخیره‌سازی نقاط

۲۰ نمره

می‌خواهیم مجموعه‌ای از نقاط را ذخیره کنیم که هر نقطه به صورت (x, y) است و:

$$0 \leq x < 1000, \quad 0 \leq y < 1000$$

۱. یک تابع طراحی کنید که هر نقطه (x, y) را به یک کلید عدد صحیح یکتا $K(x, y)$ تبدیل کند.

۲. اگر جدول هش m خانه داشته باشد، توابع هش اولیه و ثانویه زیر را تعریف کنید:

$$h_1(K) = ? \quad h_2(K) = ?$$

به طوری که:

- $h_2(K)$ هیچ وقت صفر نباشد،
- و $h_2(K)$ نسبت به m اول باشد.

۳. اگر تابع هش دوم نسبت به m اول نباشد چه مشکلاتی ممکن است پیش بیاید؟ و اگر $h_2(K) = 0$ شود، چه اتفاقی می‌افتد؟

۴. شبه کد برای توابع $INSERT(x, y)$ و $SEARCH(x, y)$ با استفاده از توابع h_1 و h_2 که در قسمت قبلی تعریف کرده‌اید بنویسید.

پاسخ:

۱. تبدیل مختصات به کلید یکتا:

چون $0 \leq y < 1000$ ، می‌توانیم از این ویژگی استفاده کنیم تا y در سه رقم پایین و x در ارقام بالاتر قرار گیرد. این روش شبیه سیستم اعداد در مبنای ۱۰۰۰ است. تابع $K(x, y)$ به صورت زیر تعریف می‌شود:

$$K(x, y) = x \times 1000 + y$$

این تابع یک تناظر یک‌به‌یک است و برای هر جفت (x, y) در بازه‌ی داده شده، یک عدد صحیح یکتا در بازه‌ی $[0, 999999]$ تولید می‌کند.

۲. تعریف توابع هش:

برای استفاده از روش Double Hashing، معمولاً فرض می‌کنیم که m یک عدد اول است. توابع زیر را پیشنهاد می‌دهیم:

$$h_1(K) = K \bmod m$$

$$h_2(K) = 1 + (K \bmod (m - 1))$$

بررسی ویژگی‌ها:

- غیر صفر بودن: چون باقی‌مانده‌ی تقسیم بر $m - 1$ همواره عددی نامنفی است (≥ 0)، پس $1 + (K \bmod (m - 1)) \geq 1$ همواره ≥ 1 خواهد بود و هرگز صفر نمی‌شود.
- اول بودن نسبت به m : اگر m عدد اول باشد، هر عددی که کوچکتر از m باشد (و بزرگتر از صفر)، نسبت به m اول است. خروجی h_2 عددی در بازه‌ی $[1, m - 1]$ است، بنابراین شرط $\gcd(h_2(K), m) = 1$ برقرار خواهد بود.

۳. تحلیل مشکلات احتمالی:

• اگر $h_r(K)$ نسبت به m اول نباشد: در این صورت هش ما تمام خانه‌های جدول را پوشش نمی‌دهد. کاوش تنها روی خانه‌هایی انجام می‌شود که مضاربی از $d = \gcd(h_r(K), m)$ باشند. یعنی تنها $\frac{m}{d}$ خانه بررسی می‌شود و سپس سیکل تکرار می‌شود. نتیجه این است که ممکن است جدول خانه‌های خالی داشته باشد، اما الگوریتم بتواند آن‌ها را پیدا کند و درج با شکست مواجه شود.

• اگر $h_r(K) = 0$ باشد: گام حرکت صفر می‌شود. یعنی دنباله‌ی اندیس‌ها به صورت $h_1, h_1, h_1, \dots$ در می‌آید. اگر در خانه‌ی اولیه $h_1(K)$ تصادم رخ داده باشد، الگوریتم وارد یک حلقه‌ی بی‌نهایت می‌شود و دائماً همان خانه‌ی پر را چک می‌کند.

۴. شبه‌کد عملیات:

فرض می‌کنیم جدول هش T دارای m خانه است. هر خانه می‌تواند وضعیت‌های EMPTY، OCCUPIED یا DELETED داشته باشد.

عملیات $INSERT(x, y)$:

```
function INSERT(x, y):
    key = x * 1000 + y
    val_h1 = key % m
    val_h2 = 1 + (key % (m - 1))

    for i from 0 to m - 1:
        index = (val_h1 + i * val_h2) % m

        if T[index] is EMPTY or T[index] is DELETED:
            T[index] = key
            return SUCCESS

        if T[index] == key:
            return ALREADY_EXISTS

    return TABLE_FULL_ERROR
```

عملیات $SEARCH(x, y)$:

```
function SEARCH(x, y):
    key = x * 1000 + y
    val_h1 = key % m
    val_h2 = 1 + (key % (m - 1))

    for i from 0 to m - 1:
        index = (val_h1 + i * val_h2) % m

        if T[index] is EMPTY:
            return NOT_FOUND

        if T[index] == key:
            return FOUND at index

    return NOT_FOUND
```

۶. احتمال تصادم

۱۵ نمره

فرض کنید یک جدول هش با m خانه داریم و قرار است تعداد n کلید متمایز $k_1, \dots, k_n$ را در این جدول ذخیره کنیم. تابع هش h به گونه‌ای عمل می‌کند که برای هر کلید k_i ، خانه‌ی مقصد $h(k_i)$ را به صورت کاملاً تصادفی و مستقل از بین اندیس‌های $\{0, \dots, m-1\}$ انتخاب می‌کند.

۱. با فرض اینکه $m = n^2$ باشد، فرمول دقیق احتمال این که هیچ گونه برخوردی رخ ندهد را بنویسید. (برخورد زمانی رخ می‌دهد که دو کلید متفاوت به یک خانه ارجاع داده شوند).

۲. در حالت کلی (بدون فرض قسمت قبل)، مقدار n بر حسب m باید حداقل چقدر باشد تا احتمال وقوع «حداقل یک برخورد» بیشتر از ۵۰ درصد شود؟ (برای محاسبه‌ی ضرایب عددی دقیق، می‌توانید از ماشین حساب یا اینترنت استفاده کنید).

پاسخ:

۱. محاسبه احتمال عدم برخورد:

می‌خواهیم احتمال این که تمام n کلید در خانه‌های متمایز قرار بگیرند را محاسبه کنیم. فرض کنید کلیدها را یک‌به‌یک وارد جدول می‌کنیم:

- کلید اول می‌تواند در هر یک از m خانه قرار بگیرد (احتمال $\frac{m}{m} = 1$).
- کلید دوم باید در یکی از $m - 1$ خانه‌ی باقی‌مانده قرار گیرد (احتمال $\frac{m-1}{m}$).
- کلید سوم باید در یکی از $m - 2$ خانه‌ی باقی‌مانده قرار گیرد (احتمال $\frac{m-2}{m}$).
- ...
- کلید i -ام باید در خانه‌ای غیرتکراری قرار گیرد (احتمال $\frac{m-(i-1)}{m}$).

بنابراین احتمال عدم برخورد برابر است با حاصل ضرب این احتمالات:

$$P = 1 \times \left(1 - \frac{1}{m}\right) \times \left(1 - \frac{2}{m}\right) \times \dots \times \left(1 - \frac{n-1}{m}\right)$$

این رابطه را می‌توان به صورت ترکیب نیز نوشت:

$$P = \frac{m \cdot (m-1) \cdot \dots \cdot (m-n+1)}{m^n} = \frac{m!}{m^n (m-n)!}$$

با جایگذاری $m = n^2$:

$$P = \prod_{i=0}^{n-1} \left(1 - \frac{i}{n^2}\right)$$

(نکته: برای n های بزرگ، این مقدار به $0.606 \approx e^{-1/2}$ میل می‌کند).

۲. محاسبه n برای احتمال برخورد بالای ۵۰ درصد:

احتمال وقوع «حداقل یک برخورد» برابر است با $1 - P$. می‌خواهیم این احتمال بیشتر از ۰/۵ باشد، که معادل است با:

$$P < 0.5$$

برای محاسبه، از تقریب $1 - x \approx e^{-x}$ (وقتی x کوچک است) استفاده می‌کنیم:

$$P \approx e^{-1} \cdot e^{-\frac{1}{m}} \cdot e^{-\frac{2}{m}} \dots e^{-\frac{n-1}{m}} = e^{-\frac{1}{m} \sum_{i=0}^{n-1} i}$$

از آنجا که $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$ داریم:

$$P \approx e^{-\frac{n^2}{2m}}$$

حال نامساوی را حل می‌کنیم:

$$e^{-\frac{n^2}{2m}} < 0.5 \implies -\frac{n^2}{2m} < \ln(0.5) = -\ln(2)$$

$$\frac{n^2}{2m} > \ln(2) \implies n^2 > 2m \ln(2) \implies n > \sqrt{2 \ln(2) \cdot m}$$

با جایگذاری مقدار تقریبی $\ln(2) \approx 0.693$:

$$n > \sqrt{2 \cdot (0.693) \cdot m} \approx \sqrt{1.386m} \approx 1.177\sqrt{m}$$

بنابراین تعداد کلیدها باید تقریباً $1.177\sqrt{m}$ باشد (کمی بیشتر از جذر تعداد خانه‌ها) تا احتمال تصادم از ۵۰ درصد عبور کند.